

QuerySage Architecture

Capstone Project — Kaggle 5-Day AI Agents Vibe Coding Course

Executive Summary

QuerySage is a natural-language-to-SQL analytics platform built with Google ADK 2.0. Users type business questions in plain English; the system validates, translates to SQL, executes against a Neon Postgres cloud database, and returns results with Plotly charts — all with human-in-the-loop approval before any query touches the database.

System Architecture

The platform is composed of three FastAPI micro-services plus a Streamlit front-end. The **Master Orchestrator** (port 8000) runs the ADK 2.0 graph workflow and coordinates calls to the **Gatekeeper** (port 9000) for regex-based input sanitisation and to the **SQL Engine** (port 9001) for Gemini-powered SQL generation. The Streamlit UI (port 8501) provides the chat interface with HITL approval buttons and Plotly visualisations.

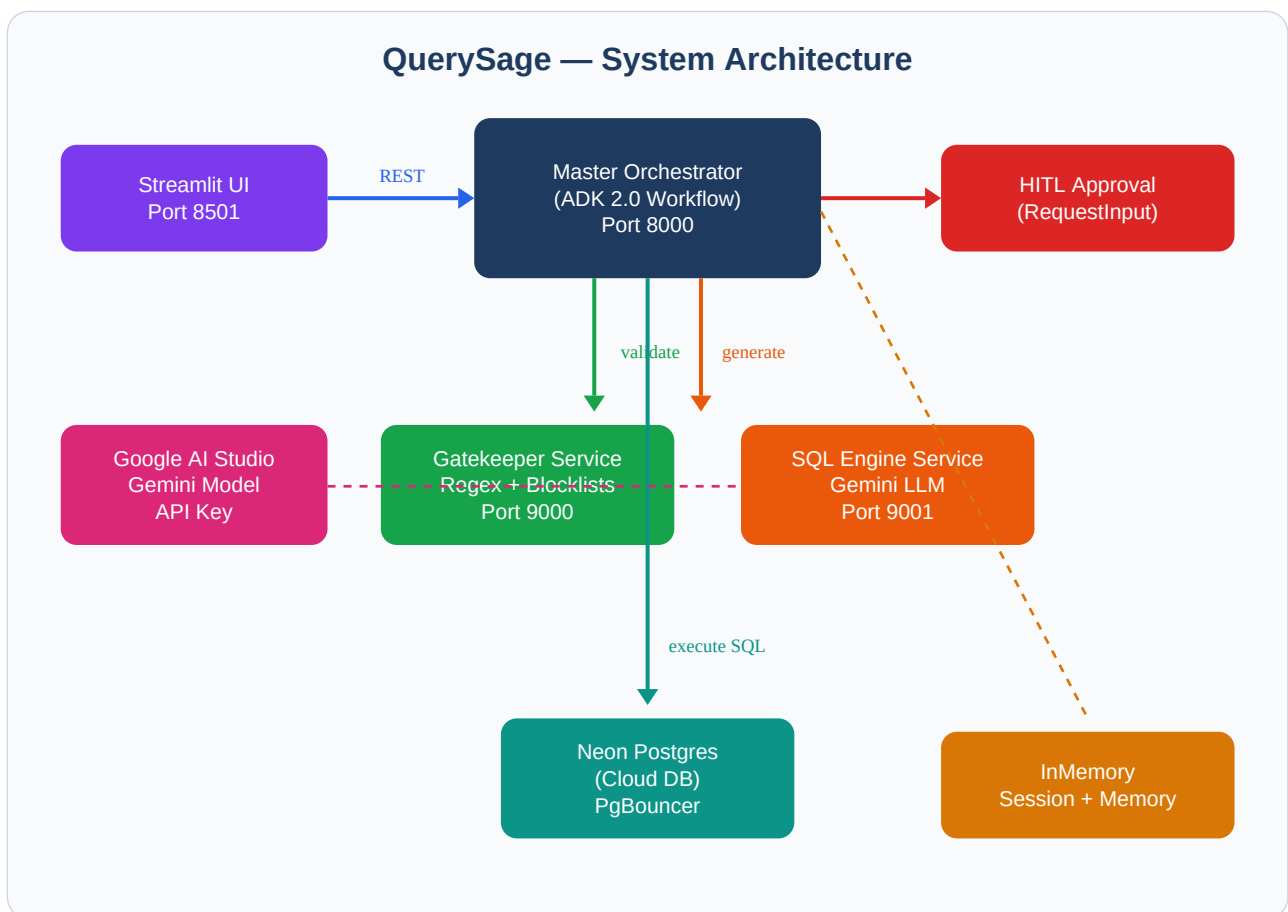


Figure 1 — System architecture showing service interactions, ports, and data flow.

Service	Port	Technology	Role
Master Orchestrator	8000	FastAPI + ADK 2.0	Graph workflow engine, HITL, session mgmt

Gatekeeper	9000	FastAPI + Regex	Input validation, SQL-injection blocking
SQL Engine	9001	FastAPI + Gemini	NL-to-SQL translation via LLM
Streamlit UI	8501	Streamlit + Plotly	Chat interface, charts, approval buttons
Neon Postgres	Cloud	PgBouncer + SSL	Analytics database (e-commerce data)

ADK 2.0 Graph Workflow

The master orchestrator runs a 9-node directed graph built with Google ADK 2.0's **GraphFlow** API. Each node is a Python function that reads/writes shared state. The workflow enforces a strict pipeline: receive → validate → generate → security-check → approve (HITL) → execute → analyse → present.

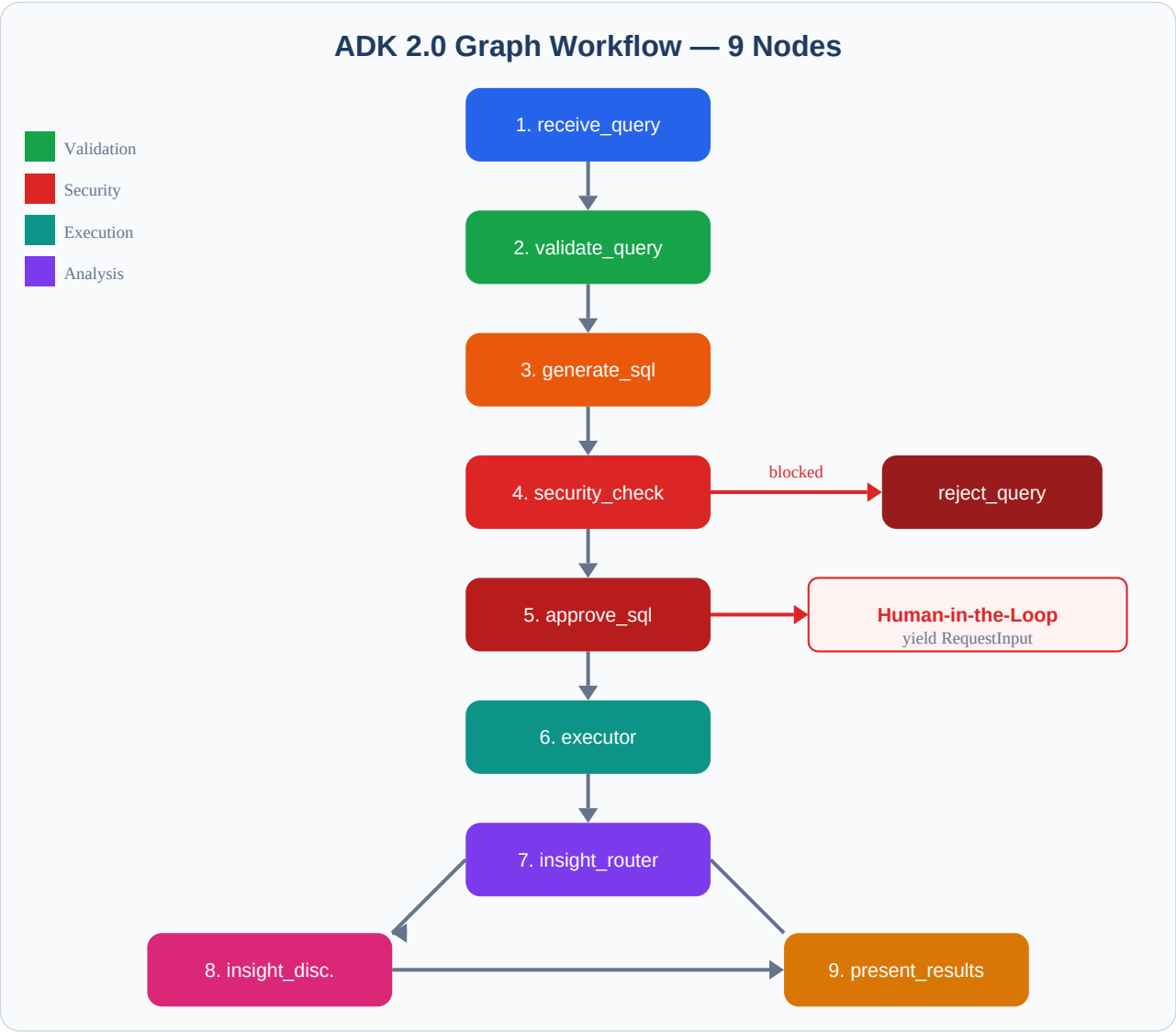


Figure 2 — The 9-node ADK 2.0 graph workflow with HITL approval gate.

#	Node	Description
1	receive_query	Accepts the NL question from the user and stores it in state
2	validate_query	Calls Gatekeeper service for regex/blocklist validation
3	generate_sql	Calls SQL Engine (Gemini LLM) to translate NL to SQL
4	security_checkpoint	Final safety check — blocks DROP/DELETE/UPDATE mutations
5	approve_sql	HITL gate — yields RequestInput, waits for user yes/no
6	executor	Runs the approved SQL against Neon Postgres via neon_tools
7	insight_router	Routes to insight discovery or directly to presentation
8	insight_discovery	Uses Gemini to generate analytical insights from results

9	present_results	Formats the final response with data + insights
---	-----------------	---

7 Kaggle Course Concepts

QuerySage demonstrates all seven concepts taught in the Kaggle 5-Day AI Agents Vibe Coding Course. The diagram below shows how each concept connects to the central QuerySage platform.



Figure 3 — The 7 course concepts and how QuerySage implements each.

#	Concept	Implementation
1	Graph Workflow	ADK 2.0 GraphFlow with 9 sequential + branching nodes
2	Multi-Agent	3 independent FastAPI services communicating via REST
3	HITL (Human-in-the-Loop)	RequestInput at approve_sql node; FunctionResponse resume
4	Tool Use	neon_execute_sql() and generate_chart() registered as ADK tools
5	Safety Guardrails	Regex blocklists, sqlparse validation, SELECT-only enforcement
6	Memory	InMemoryMemoryService persists context across conversation turns
7	Context Engine	EventsCompactionConfig + ResumabilityConfig for state management

Human-in-the-Loop (HITL) — Deep Dive

The HITL mechanism is the most critical safety feature. When the workflow reaches the **approve_sql** node, it yields a **RequestInput** object. ADK 2.0 converts this into an Event containing a **FunctionCall** with name `adk_request_input` and the `interrupt_id` "approved".

The master's FastAPI endpoint detects this by scanning event content parts for `part.function_call.name == "adk_request_input"`. It returns `status: "interrupted"` to the Streamlit UI, which shows Approve/Reject buttons. When the user clicks Approve, the UI sends "yes" back, and the master resumes the workflow by sending a **FunctionResponse** with the matching `interrupt_id`.

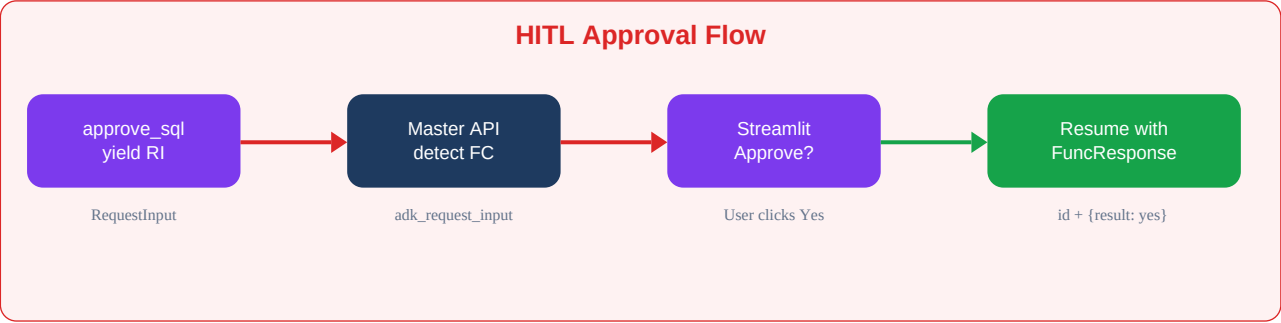


Figure 4 — HITL approval flow from node to UI and back.

Security Layers

QuerySage implements defence-in-depth with multiple security layers:

Layer	Component	Mechanism
1 — Input	Gatekeeper (port 9000)	Regex patterns block SQL injection, XSS, system commands
2 — Generation	SQL Engine (port 9001)	Gemini prompt instructs SELECT-only generation
3 — Validation	security_checkpoint node	sqlparse tokenisation rejects non-SELECT statements
4 — Approval	approve_sql node	Human reviews exact SQL before execution
5 — Execution	neon_tools.py	Blocked keyword set (DROP, DELETE, UPDATE, etc.)
6 — Database	Neon Postgres	Read-only connection role, SSL + channel_binding

Technology Stack

Category	Technology	Details
AI Framework	Google ADK 2.0	GraphFlow, RequestInput, Runner, Events
LLM	Google Gemini	gemini-2.0-flash via Google AI Studio API key
Backend	FastAPI + Uvicorn	3 async micro-services
Frontend	Streamlit + Plotly	Interactive chat UI with charts
Database	Neon Postgres	Serverless, auto-suspend, PgBouncer pooling
SQL Parsing	sqlparse	Token-level SQL validation
HTTP Client	httpx	Async inter-service communication
Environment	python-dotenv	.env-based configuration

Quick Start

1. Clone the repo and install dependencies: `pip install -r requirements.txt`
2. Copy `.env.example` to `.env` and fill in `GOOGLE_API_KEY` and `NEON_DATABASE_URL`.
3. Start the three services:

```
uvicorn app.fast_api_app:app --port 8000
uvicorn services.gatekeeper.gatekeeper_app:app --port 9000
uvicorn services.sql_engine.sql_engine_app:app --port 9001
```
4. Launch the Streamlit UI: `streamlit run streamlit_app.py`
5. Open `http://localhost:8501`, type a question, approve the SQL, and see results with charts.